

KUPC Phase 5 Specification

Student & Company Profiles + Frontend Integration Foundation

Version 1.1 · July 17, 2026 · Status: Draft for sprint planning · Detailed milestone steps

Document Control

Field	Value
Project	KUPC — Kathmandu University Placement Connect
Phase	5 (recommended — supersedes deferred v2 Phase 3 profile work)
Depends on	Phase 2 (Auth), Phase 3B (DB/repos), Phase 4 (Resumes)
Feeds into	Phase 6 — Job Posting & Discovery (v2 spec Phase 5)
Repos	kupc-backend + frontend (monorepo)
Audience	Backend engineers, frontend engineers, technical reviewers

1. Phase Goal

Make the product **usable end-to-end** for authenticated students and companies by delivering profile APIs on the backend and wiring the existing frontend prototype to real backend services — starting with auth, profiles, and resume analysis.

At the end of Phase 5, a student can register, verify OTP, log in, view/edit their profile, upload a resume, and see real AI analysis results. A company can log in, view/edit its profile (pending verification), and see verification status. The frontend no longer relies on hardcoded mock data for these flows.

2. Why This Phase (Not Job Posting Yet)

- Backend Specification v2.0 lists Job Posting as Phase 5, but v2 Phase 3 (Profiles) was never implemented as APIs.
- Phase 4 internal docs explicitly target Profiles as the next backend milestone.
- Frontend has 0% API integration — job endpoints would have no consumer.
- Job posting requires company identity, profile metadata, and requireVerifiedCompany middleware context.
- Resume analysis (completed backend) is the highest-value feature to wire first.

3. Milestone Overview

ID	Side	Title	Exit criterion
B1	Backend	Students profile module	GET/PATCH /students/me + public GET pass tests
B2	Backend	Companies profile module	GET/PATCH /companies/me + public GET pass tests
B3	Backend	Avatar & logo storage	Uploads land in Supabase; URLs persisted
B4	Backend	Swagger, errors, hardening	OpenAPI paths live; error codes stable

ID	Side	Title	Exit criterion
B5	Backend	Phase 5 test matrix	npm run test:phase5 green; phase4 still green
F1	Frontend	API client & env	Authenticated requests reach /api/v1
F2	Frontend	Token & session layer	Refresh on 401; session survives reload
F3	Frontend	Auth screens	Register → OTP → login for all three roles
F4	Frontend	Guards & role routing	Wrong role cannot open protected screens
F5	Frontend	Student profile wiring	Live GET/PATCH replaces STUDENTS[0]
F6	Frontend	Company profile wiring	Placeholder replaced; live CRUD
F7	Frontend	Resume analyzer wiring	Real upload + poll + score UI
F8	Frontend	UX polish & cleanup	Loading/error/empty; mock data scoped

3.1 Out of Scope

- Job CRUD, search/filter, saved jobs (Phase 6)
- Swipe engine, matching, chat, notifications
- Admin company-approval API workflow (v2 Phase 11)
- pgvector recommendations
- Full App.tsx → react-router rewrite (optional stretch only)

3.2 Suggested Order

B1 → B2 → B3 → B4 → B5 in parallel with F1 → F2 → F3 → F4. Then F5 and F6 (need B1–B3). Then F7 (needs Phase 4 resumes + F2). Finish with F8.

4. API Contract (Backend)

Method	Path	Auth	Description
GET	/api/v1/students/me	Student	Own profile + linked resume summary
PATCH	/api/v1/students/me	Student	bio, phone, degree, cgpa, department, graduation_year
POST	/api/v1/students/me/avatar	Student	Multipart image; returns profile_picture_url
GET	/api/v1/students/:id	Authenticated	Public student card (no phone / KU id)
GET	/api/v1/companies/me	Company	Own profile incl. verification_status
PATCH	/api/v1/companies/me	Company	company_name, website, industry, description
POST	/api/v1/companies/me/logo	Company	Multipart logo; returns logo_url
GET	/api/v1/companies/:id	Authenticated	Approved companies only

Reuse existing auth (already live):

- POST /api/v1/auth/register/student, /register/company, /verify-otp
- POST /api/v1/auth/login, /admin/login, /refresh, /logout
- GET /api/v1/auth/me
- Resume APIs (Phase 4): POST/GET/DELETE /api/v1/resumes, GET /:id/analysis

5. Backend Milestones — Detailed Steps

Follow the Phase 4 resumes module layout: `src/modules/<name>/` with types, validation, service, controller, routes, mapper, errors, constants, index. Mount under `src/routes/index.ts` at `/api/v1`. Repositories already exist in `src/database/`.

Milestone B1 — Students Profile Module

Goal: Expose authenticated student profile read/update and a public card endpoint. **Depends on:** Phase 2 auth middleware, `studentsRepository`.

Backend steps

1. Create folder `src/modules/students/` mirroring resumes module structure.
2. Add `students.types.ts`: `StudentProfileDto`, `StudentPublicCardDto`, `UpdateStudentProfileInput` (camelCase API shape).
3. Add `students.validation.ts` with Zod: PATCH body (optional string/number fields, `cgpa` 0–4, `graduation_year` integer), params schema for `:id` (UUID).
4. Add `students.errors.ts`: `STUDENT_NOT_FOUND`, `STUDENT_PROFILE_FORBIDDEN`, `INVALID_PROFILE_PAYLOAD` — map to `AppError` with stable codes.
5. Add `students.mapper.ts`: map `StudentRecord` → `StudentProfileDto`; public mapper that **omits** phone and `ku_id`.
6. Add `students.service.ts`:
 - `getMe(userId)`: `studentsRepository.findById`; throw `STUDENT_NOT_FOUND` if null; optionally join active resume summary via `resumesRepository` if `resume_id` set.
 - `updateMe(userId, input)`: call `studentsRepository.updateProfile`; return mapped DTO.
 - `getPublicById(id)`: `findById`; return public mapper (for company discover later).
1. Add `students.controller.ts`: thin handlers reading `req.user.id`, calling service, wrapping with `successResponse`.
2. Add `students.routes.ts`:
 - Router-level: authenticate on all routes.
 - GET `/me` + PATCH `/me`: `authorize(Role.STUDENT)`.
 - GET `/:id`: `authorize` any authenticated role (`STUDENT | COMPANY | ADMIN`).
 - Register PATCH before `/:id` so 'me' is never captured as an id.
1. Export from `students/index.ts` and mount: `router.use('/students', studentsRouter)` in `src/routes/index.ts`.
2. Manual smoke: login as student → GET `/students/me` → PATCH bio → GET again.

Exit criteria B1

- GET `/students/me` returns 200 with profile for student JWT; 401 without token; 403 for company JWT.
- PATCH updates only allowed fields; invalid `cgpa` returns 400 with stable error code.
- GET `/students/:id` returns public card without phone/`ku_id`.

Milestone B2 — Companies Profile Module

Goal: Company self-service profile + public approved-company card. **Depends on:** B1 patterns, `companiesRepository`.

Backend steps

1. Create `src/modules/companies/` (types, validation, errors, mapper, service, controller, routes, index) — copy structure from `students`.
2. DTO must include `verification_status` and `verified_at` on `/me` (companies need to see pending state).

3. Validation: PATCH allows companyName (min 2), website URL optional, industry, description (max length).
4. Service getMe(userId): companiesRepository.findByUserId.
5. Service getPublicById(id): findById; if verification_status !== 'approved', throw NOT_FOUND (do not leak pending companies to students).
6. Service updateMe: companiesRepository.updateProfile — never allow client to set verification_status.
7. Routes: authenticate all; GET/PATCH /me authorize(Role.COMPANY); GET /:id any authenticated role.
8. Mount at /companies in routes/index.ts.
9. Smoke: company login → GET /companies/me shows pending → PATCH description → confirm.

Exit criteria B2

- Company can read/update own profile; cannot change verification_status via PATCH.
- Public GET returns 404 for pending/rejected companies.
- Student JWT cannot PATCH /companies/me (403).

Milestone B3 — Avatar & Logo Storage Uploads

Goal: Multipart image upload for student avatar and company logo, stored in Supabase Storage, URL written to profile rows. **Pattern:** Phase 4 `resumeStorage.ts` + `multer` middleware.

Backend steps

1. In Supabase dashboard (or SQL migration notes), create storage buckets `avatars` and `company-logos` with public read; uploads only via service role from the API.
2. Add env keys if needed (bucket names) to `src/config/env.ts` Zod schema with defaults matching Phase 4 `RESUME_*` style.
3. Create `src/services/profileImageStorage.ts` (or under each module): `uploadAvatar(userId, buffer, mime)`, `uploadCompanyLogo(companyId, buffer, mime)` — path like `{userId}/{timestamp}.jpg`.
4. Add `multer` middleware: accept `image/jpeg`, `image/png`, `image/webp`; max 2 MB; single field `file` or `avatar / logo`.
5. Students: `POST /students/me/avatar` → upload → `studentsRepository.updateProfile({ profilePictureUrl })` → return updated profile DTO.
6. Companies: `POST /companies/me/logo` → same pattern with `logo_url`.
7. On replace: optionally delete previous object from storage (best-effort; do not fail the request).
8. Map `multer` errors to `AppError` (`INVALID_FILE_TYPE`, `FILE_TOO_LARGE`) like `resumes` upload utils.
9. Add light rate limiter on upload routes (reuse or clone `resumeUploadRateLimiter` pattern).

Exit criteria B3

- Valid JPEG/PNG returns 200 with new URL; URL is publicly fetchable.
- PDF or >2 MB returns 400 with stable code.
- `students.profile_picture_url` / `companies.logo_url` updated in DB.

Milestone B4 — Swagger, Error Catalog & Hardening

Backend steps

1. Extend `src/config/swagger.ts` paths for every `students/companies` endpoint (request/response schemas, bearer security, error examples).
2. Document error codes in `PHASE_5_DOCUMENTATION.md` (create file): `STUDENT_NOT_FOUND`, `COMPANY_NOT_FOUND`, `COMPANY_NOT_PUBLIC`, `INVALID_FILE_TYPE`, etc.
3. Confirm CORS allows the Vite origin (`localhost:5173`) for Authorization header.
4. Ensure PATCH does not accept unknown keys (Zod `.strict()` or `strip`).
5. Double-check route order: `/me` and `/me/avatar` registered before `/:id`.
6. Run `npm run typecheck`; fix any new module export issues.

Exit criteria B4

- Swagger UI at `/api/docs` lists all Phase 5 endpoints with try-it-out schemas.
- Typecheck passes; no unused exports from new modules.

Milestone B5 — Phase 5 Test Matrix

Backend steps

1. Add `src/__tests__/phase5.students.test.ts`: auth matrix, happy path GET/PATCH, validation failures, public card field stripping.
2. Add `src/__tests__/phase5.companies.test.ts`: pending company not public, approved public, company-only PATCH, student forbidden.
3. Add `src/__tests__/phase5.upload.test.ts`: mock storage upload; assert repository update called with URL; reject bad mime.

4. Add `src/__tests__/phase5.swagger.test.ts`: assert path keys exist on `swaggerSpec` (mirror `phase4.swagger.test.ts`).
5. Add npm script `test:phase5` in `package.json`: `jest --runInBand src/__tests__/phase5`.
6. Run `npm run test:phase5` and `npm run test:phase4` — both must pass.
7. Write `documentation/PHASE_5_DOCUMENTATION.md` with milestone table `Status=Complete` as each lands.

Exit criteria B5

- `npm run test:phase5` exits 0.
- `npm run test:phase4` still green (no regressions).
- `PHASE_5_DOCUMENTATION.md` checked in with milestone status.

6. Frontend Milestones — Detailed Steps

Frontend root: `c:/projects/KU-Placement-Cell/frontend`. Today everything lives in `src/app/App.tsx` (~2100 lines) with `useState` screen switching and hardcoded COMPANIES / STUDENTS / MESSAGES. Phase 5 does **not** require a full rewrite — add an integration layer beside `App.tsx` and replace only the wired screens.

Milestone F1 — API Client & Environment

Goal: One place for all HTTP calls to the backend. **Depends on:** Backend running locally (default port from Phase 1 env).

Frontend steps

1. Create `frontend/.env.example` and `.env` with `VITE_API_URL=http://localhost:5000/api/v1`.
2. Create `src/lib/api/client.ts`:
 - read base URL from `import.meta.env.VITE_API_URL`.
 - `apiRequest<T>(path, options): fetch, JSON parse, throw ApiError on non-OK`.
 - Parse backend envelope `{ success, data, message, error }` — return data on success.
 - Support body: JSON (default) and FormData (skip Content-Type so browser sets boundary).
3. Create `src/lib/api/types.ts` for shared DTO types (`AuthTokens`, `StudentProfile`, `CompanyProfile`, `ResumeAnalysis`).
4. Create thin API modules: `authApi.ts`, `studentsApi.ts`, `companiesApi.ts`, `resumesApi.ts` — one function per endpoint.
5. Verify CORS: from browser console or a temporary button, call `GET /health` or unauthenticated register validation.
6. Confirm Vite proxies are **not** required if CORS is correct; document either approach.

Exit criteria F1

- Importing `studentsApi` from a throwaway call reaches the backend without CORS errors.
- `.env.example` committed; secrets never committed.

Milestone F2 — Token Storage & Session Layer

Frontend steps

1. Create `src/lib/auth/tokenStore.ts`: `get/set/clear accessToken + refreshToken` in `localStorage` (keys e.g. `kupc_access`, `kupc_refresh`).
2. Extend api client: if access token present, set `Authorization: Bearer ....`
3. On HTTP 401: attempt single refresh via `POST /auth/refresh` with refresh token; store new pair; retry original request once; if refresh fails, clear tokens and emit session-expired event.
4. Create `src/lib/auth/AuthContext.tsx`:
 - State: `user (from /auth/me), role, status: idle | loading | authenticated | anonymous`.
 - `login(email, password) → authApi.login → save tokens → fetchMe()`.
 - `logout() → authApi.logout (best-effort) → clear tokens → set anonymous`.
 - On mount: if access token exists, call `/auth/me`; else anonymous.
5. Wrap App root with `AuthProvider` in `src/main.tsx` (or wherever `createRoot` is).
6. Expose `useAuth()` hook for screens.

Exit criteria F2

- After manual token inject + reload, `AuthContext` hydrates user from `/auth/me`.
- Expired access token triggers refresh without forcing re-login when refresh is valid.

Milestone F3 — Auth Screens

Goal: Real login/register/OTP UI. Landing page currently jumps straight into the app via `onEnter(role)` — replace that with auth flows.

Frontend steps

1. Add Screen union values (or separate auth view state): login, register-student, register-company, verify-otp, admin-login.
2. Build LoginPage: email + password → `authApi.login` → on success set role from `/me` and navigate to dashboard.
3. Build RegisterStudentPage: fields matching `registerStudentSchema` (KU email, password, full name, `ku_id`) → `POST /auth/register/student` → navigate to verify-otp with email in state.
4. Build VerifyOtpPage: 6-digit OTP → `POST /auth/verify-otp` → then login or accept tokens if API returns them → dashboard.
5. Build RegisterCompanyPage: `company_name`, email, password, website → `POST /auth/register/company` → login screen with success banner (pending verification messaging).
6. Build AdminLoginPage: `POST /auth/admin/login`.
7. Update LandingPage CTAs: Student / Company / Admin open the correct auth screen (not fake dashboard entry).
8. Surface backend error codes as readable messages (`EMAIL_ALREADY_EXISTS`, `INVALID_OTP`, `INVALID_CREDENTIALS`, etc.).
9. Disable submit buttons while request in flight; show inline field errors.

Exit criteria F3

- Student can register → verify OTP → land on student dashboard with real JWT.
- Company can register → login → see company shell (verification pending).
- Admin can login via admin route.
- Wrong password shows error; no silent failure.

Milestone F4 — Guards & Role Routing

Frontend steps

1. In App shell: if `AuthContext` status is loading, show full-page spinner (not landing).
2. If anonymous and screen is not an auth/landing screen, redirect to login.
3. If authenticated, set role from `user.role` (`STUDENT`→student, `COMPANY`→company, `ADMIN`→admin) — do not trust the landing-page role picker alone.
4. Filter sidebar NAV items by role (already partially done); additionally block `screenComponent` access if role mismatch (e.g. student opening company-approval → redirect dashboard).
5. On logout from Settings: call `AuthContext.logout` and return to landing.
6. Optional: deep-link query `?screen=profile` after login for later phases.

Exit criteria F4

- Unauthenticated user cannot remain on dashboard after refresh.
- Student JWT cannot open admin screens via `setScreen` hacking.
- Logout clears tokens and returns to landing.

Milestone F5 — Wire Student Profile Screen

Depends on: B1, F2. Target component: `StudentProfile` in `App.tsx` (currently reads `STUDENTS[0]`).

Frontend steps

1. On mount of StudentProfile: `studentsApi.getMe()` → local form state (`full_name` read-only or editable per backend rules; `bio`, `phone`, `degree`, `cgpa`, `department`, `graduation_year` editable).
2. Show skeleton/spinner while loading; toast or banner on load error.
3. Save button → `studentsApi.updateMe(patch)` → update local state from response.
4. Avatar: file input → `studentsApi.uploadAvatar(file)` → refresh profile image src.
5. Display linked resume summary if API returns `resume_id` / analysis score (link button to resume screen).
6. Remove `STUDENTS[0]` usage from this screen and from sidebar avatar when authenticated (use `profile_picture_url` or initials fallback).
7. Keep SettingsPage personal fields in sync or make Settings a thin alias that navigates to profile (avoid two sources of truth).

Exit criteria F5

- Profile shows live backend data after login (not Priya mock).
- Editing bio and saving persists after full page reload.
- Avatar upload updates the visible image.

Milestone F6 — Wire Company Profile Editor

Depends on: B2–B3, F2. Target: replace the company-profile placeholder block in App.tsx.

Frontend steps

1. Create CompanyProfileEditor component (can live in App.tsx or src/app/screens/CompanyProfile.tsx).
2. On mount: companiesApi.getMe() → form fields company_name, website, industry, description.
3. Show verification_status badge: pending (amber), approved (green), rejected (red) — copy explaining that job posting requires approval.
4. Save → companiesApi.updateMe; logo file input → companiesApi.uploadLogo.
5. Disable logo/name edit messaging if product rules require lock after approval (default: allow description/industry always; document any locks).
6. Wire screenComponent['company-profile'] to the new editor.
7. Company dashboard header: show real company_name from context/me instead of mock.

Exit criteria F6

- Placeholder text is gone; form loads and saves against API.
- Pending companies see clear verification status.
- Logo upload displays after save.

Milestone F7 — Wire Resume Analyzer

Depends on: Phase 4 backend + worker running (Redis, OPENAI_API_KEY), F2. Target: ResumeAnalyzer — currently fake setTimeout(~2s).

Frontend steps

1. Replace mock analyze handler: on PDF select/upload, call resumesApi.upload(file) expecting 202 with resumeld + analysisid + status pending.
2. Set UI to analyzing state immediately on 202.
3. Poll resumesApi.getAnalysis(resumeld) every 2 seconds (max ~60 attempts).
4. On status completed: map response into existing ScoreRing / breakdown / strengths / suggestions UI (align field names with resumes.mapper / INTEGRATION-inspired shape).
5. On status failed: show error message from API; allow retry upload.
6. List prior resumes via resumesApi.list() in a side panel or dropdown (optional but recommended).
7. Delete resume: confirm dialog → resumesApi.remove(id) → refresh list.
8. Clear the fake timeout and hardcoded score constants from the component.
9. Document in README: both npm run dev (API) and npm run worker:resumes must run for analysis to complete.

Exit criteria F7

- Uploading a real PDF produces a live ATS score from OpenAI (worker path).
- Pending → processing → completed states visible in UI.
- Failed analysis shows recoverable error.

Milestone F8 — UX Polish & Mock-Data Cleanup

Frontend steps

1. Add shared LoadingSpinner, ErrorBanner, EmptyState components; use on all wired screens.
2. Global toast or top banner for network failures (offline, 500).

3. Scope mock data: keep COMPANIES/STUDENTS/MESSAGES only for discover, matches, chat, kanban, admin approval until later phases — add a code comment // MOCK: Phase 6+.
4. Ensure Settings logout and notification bell do not crash when user is real (notifications may remain mock).
5. Mobile sanity check: auth forms and profile forms usable at 375px width.
6. Write frontend/INTEGRATION.md short note: env vars, how to run backend+worker+frontend, which screens are live vs mock.
7. Manual test matrix (checklist below) executed and results noted in Phase 5 docs.

Exit criteria F8

- No STUDENTS[0] in profile/resume/auth paths.
- Discover/chat still work as demos with mocks.
- frontend/INTEGRATION.md exists.

7. Data & Storage Notes

Reuse Phase 3B tables `students` and `companies`. Repositories already implement `updateProfile`. New work is HTTP layer + storage buckets `avatars` and `company-logos`. Resume bucket from Phase 4 is unchanged.

8. Local Dev Runbook

#	Process	Command / notes
1	Supabase	Local or cloud project; migrations applied; storage buckets created
2	Redis	Required for resume worker (Phase 4)
3	Backend API	<code>cd kupc-backend && npm run dev</code>
4	Resume worker	<code>cd kupc-backend && npm run worker:resumes</code>
5	Frontend	<code>cd frontend && npm run dev (Vite)</code>
6	Docs	Swagger at <code>http://localhost:<port>/api/docs</code>

9. Testing Strategy

Backend

- supertest integration tests per endpoint (auth, validation, RBAC, not-found).
- Upload tests with mocked Supabase Storage (mirror Phase 4).
- Swagger path presence tests.
- Scripts: `test:phase5`, plus regression `test:phase4`.

Frontend (manual matrix)

#	Case	Expected
1	Student register + OTP + login	Dashboard; token in <code>localStorage</code>
2	Reload while logged in	Session restored via <code>/auth/me</code>
3	Edit student bio + reload	Bio persisted
4	Upload avatar	Image visible in profile + sidebar
5	Upload resume PDF	Score appears after worker completes
6	Company register + login	Profile shows verification pending
7	Company edit description + logo	Persists after reload
8	Admin login	Admin shell; no student routes
9	Student opens <code>/companies/me</code> via API but not 403 from API; UI never offers it	
10	Logout	Landing; tokens cleared

10. Exit Checklist (Phase Complete)

#	Criterion	Owner
1	B1–B5 complete; test:phase5 green	Backend
2	Swagger documents all Phase 5 endpoints	Backend
3	Student register → OTP → login → session persists	Frontend
4	Student profile view/edit + avatar against live API	Both
5	Resume analyzer shows real ATS score from worker	Both
6	Company login + profile edit + logo; status visible	Both
7	Guards block cross-role screens	Frontend
8	PHASE_5_DOCUMENTATION.md + frontend/INTEGRATION.md written	Both
9	test:phase4 still green	Backend

11. Phase 6 Preview — Job Posting & Discovery

After Phase 5, implement Backend Specification v2.0 Job Posting & Discovery: full jobs module (CRUD for verified companies, student feed with filters, saved jobs), wire the existing JobPosting form, and replace mock discover cards with real open jobs. jobs.repository and indexes from Phase 3B are already in place.